TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero-Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de diseño

TÍTULO:
Actividad de evidencia prototype

UNIDAD:
3

NOMBRE Y NÚMERO DE CONTROL:
Kevin Omar Alvarez Hernandez 22210280

NOMBRE DEL MAESTRO:
Maribel Guerrero Luis

Índice

Índice.....2
Introducción.....3
Código implementado:..... 4
Resultados:..... 9
Diagrama UML..... 11
Conclusión:..... 12
Referencias:..... 13

Introducción

Composite

¿Qué es?
El patrón de diseño Composite nos sirve para construir estructuras complejas partiendo de otras estructuras mucho más simples, dicho de otra manera, podemos crear estructuras compuestas las cuales están conformadas por otras estructuras más pequeñas.

Características:
Estructura en Árbol (Jerárquica)
Uniformidad
Composición Recursiva
Flexibilidad (Escalabilidad)

Ventajas:
Uniformidad y simplificación
Estructuras jerárquicas naturales
Principio Abierto/Cerrado
Mayor flexibilidad
Recursividad simplificada

Código implementado:

Program.cs

```
0 referencias
class Program
{
    0 referencias
    static void Main(string[] args)
    {
        Producto pc = new Compuesto("PC Personalizada");

        // CPU
        Producto cpu1 = new Pieza("Intel i3", 2000, "CPU", "Básico");
        Producto cpu2 = new Pieza("Ryzen 5", 3500, "CPU", "Intermedio");
        Producto cpu3 = new Pieza("Intel i9", 9000, "CPU", "Alto rendimiento");

        // RAM
        Producto ram1 = new Pieza("8GB", 800, "RAM", "Básico");
        Producto ram2 = new Pieza("16GB", 1200, "RAM", "Intermedio");
        Producto ram3 = new Pieza("32GB", 2500, "RAM", "Avanzado");

        // DISCO
        Producto d1 = new Pieza("1TB HDD", 900, "Disco", "Lento");
        Producto d2 = new Pieza("512GB SSD", 1500, "Disco", "Rápido");
        Producto d3 = new Pieza("1TB NVMe", 3000, "Disco", "Ultra rápido");

        // GPU
        Producto g1 = new Pieza("Integrada", 0, "GPU", "Incluida");
        Producto g2 = new Pieza("GTX 1660", 5000, "GPU", "Gaming medio");
        Producto g3 = new Pieza("RTX 4080", 15000, "GPU", "Gaming extremo");

        // MONITOR
        Producto m1 = new Pieza("HD", 1500, "Monitor", "Básico");
        Producto m2 = new Pieza("Full HD", 2500, "Monitor", "Bueno");
        Producto m3 = new Pieza("4K", 6000, "Monitor", "Premium");
```

```
32
33          // ARMAR PC
34          pc.Agregar(Elegir("CPU", cpu1, cpu2, cpu3));
35          pc.Agregar(Elegir("RAM", ram1, ram2, ram3));
36          pc.Agregar(Elegir("Disco", d1, d2, d3));
37          pc.Agregar(Elegir("GPU", g1, g2, g3));
38          pc.Agregar(Elegir("Monitor", m1, m2, m3));
39
40          MostrarResumen(pc);
41
42          Console.ReadLine();
43      }
44
        5 referencias
45      static Producto Elegir(string tipo, Producto a, Producto b, Producto c)
46      {
47          Console.WriteLine($"\nSeleccione {tipo}:");
48          Console.WriteLine($"1. {a.Nombre} - ${a.Precio}");
49          Console.WriteLine($"2. {b.Nombre} - ${b.Precio}");
50          Console.WriteLine($"3. {c.Nombre} - ${c.Precio}");
51          Console.Write("Opción: ");
52
53          int op;
54          if (!int.TryParse(Console.ReadLine(), out op))
55              return a;
56
57          switch (op)
58          {
59              case 1: return a;
60              case 2: return b;
61              case 3: return c;
62              default: return a;
63          }
```

```
62              default: return a;
63          }
64      }
65
        1 referencia
66      static void MostrarResumen(Producto pc)
67      {
68          Console.WriteLine("\n=== RESUMEN DE TU PC ===\n");
69
70          foreach (var item in pc.ObtenerHijos())
71          {
72              Pieza p = item as Pieza;
73
74              Console.WriteLine($"{p.Tipo}: {p.Nombre}");
75              Console.WriteLine($"Descripción: {p.Descripcion}");
76              Console.WriteLine($"Precio: ${p.Precio}\n");
77          }
78
79          Console.WriteLine($"1TOTAL: ${pc.Precio}");
80      }
81  }
```

Producto.cs

```
using System.Collections.Generic;

34 referencias
public abstract class Producto
{
    protected string nombre;

    2 referencias
    public Producto(string nombre)
    {
        this.nombre = nombre;
    }

    4 referencias
    public string Nombre => nombre;

    7 referencias
    public abstract void Agregar(Producto p);
    3 referencias
    public abstract IList<Producto> ObtenerHijos();
    8 referencias
    public abstract double Precio { get; }
}
```

Pieza.cs

```
using System.Collections.Generic;

18 referencias
public class Pieza : Producto
{
    private double precio;
    private string tipo;
    private string descripcion;

    15 referencias
    public Pieza(string nombre, double precio, string tipo, string descripcion)
        : base(nombre)
    {
        this.precio = precio;
        this.tipo = tipo;
        this.descripcion = descripcion;
    }

    1 referencia
    public string Tipo => tipo;
    1 referencia
    public string Descripcion => descripcion;

    6 referencias
    public override void Agregar(Producto p)
    {
        // No hace nada (es hoja)
    }

    2 referencias
    public override IList<Producto> ObtenerHijos()
    {
        return null;
    }

    7 referencias
    public override double Precio => precio;
}
```

[Compuesto.cs]

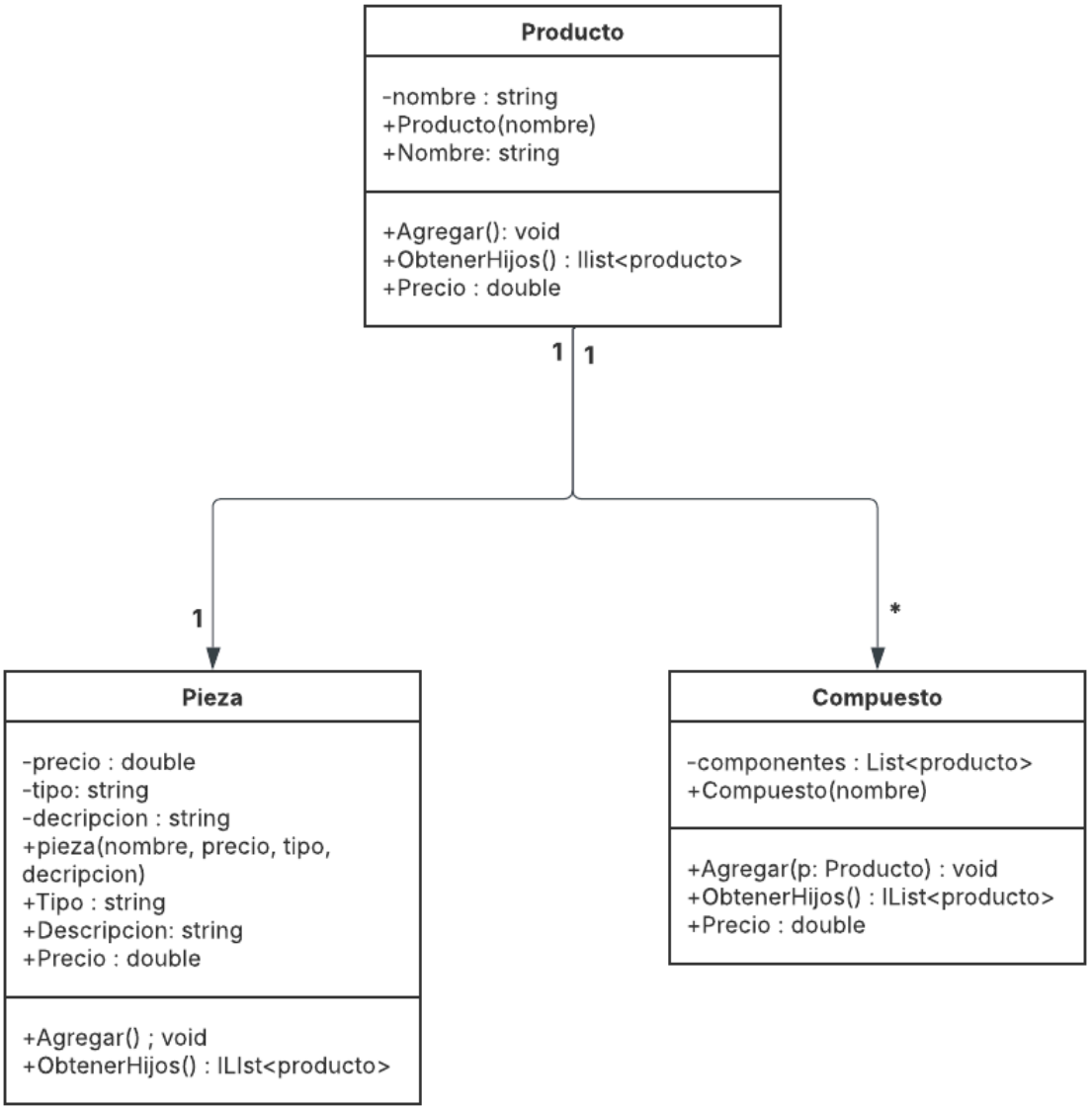
```
using System.Collections.Generic;

2 referencias
public class Compuesto : Producto
{
    private List<Producto> componentes = new List<Producto>();

    1 referencia
    public Compuesto(string nombre) : base(nombre)
    {
    }

    6 referencias
    public override void Agregar(Producto p)
    {
        componentes.Add(p);
    }

    2 referencias
    public override IList<Producto> ObtenerHijos()
    {
        return componentes;
    }

    6 referencias
    public override double Precio
    {
        get
        {
            double total = 0;
            foreach (var item in componentes)
            {
                total += item.Precio;
            }
            return total;
        }
    }
}
```

Resultados:

```
Seleccione CPU:
1. Intel i3 - $2000
2. Ryzen 5 - $3500
3. Intel i9 - $9000
Opción: 2

Seleccione RAM:
1. 8GB - $800
2. 16GB - $1200
3. 32GB - $2500
Opción: 2

Seleccione Disco:
1. 1TB HDD - $900
2. 512GB SSD - $1500
3. 1TB NVMe - $3000
Opción: 3

Seleccione GPU:
1. Integrada - $0
2. GTX 1660 - $5000
3. RTX 4080 - $15000
Opción: 2

Seleccione Monitor:
1. HD - $1500
2. Full HD - $2500
3. 4K - $6000
Opción: 2
```

```
=== RESUMEN DE TU PC ===

CPU: Ryzen 5
Descripción: Intermedio
Precio: $3500

RAM: 16GB
Descripción: Intermedio
Precio: $1200

Disco: 1TB NVMe
Descripción: Ultra rápido
Precio: $3000

GPU: GTX 1660
Descripción: Gaming medio
Precio: $5000

Monitor: Full HD
Descripción: Bueno
Precio: $2500

1TOTAL: $15200
```

Diagrama UML



Conclusión:

Es un patrón que nos permite organizar de forma jerárquica; podemos organizar mejor los elementos dentro de una lista. En forma de árbol, este nos permite ir de rama en rama y saber su peso; tiene recursión. Desde el main podemos armar la arquitectura del árbol.

Referencias:

1. *Oblancarte. (2020, 12 julio). Patrón de diseño - Composite - Oscar Blancarte - Software Architecture. Oscar Blancarte - Software Architecture. https://www.oscarblancarteblog.com/2014/10/07/patron-de-diseno-composite/*

2. *Composite. (s. f.). https://refactoring.guru/es/design-patterns/composite#:~:text=Pros%20y%20contras%20*%20Puedes%20trabajar%20con,ahora%20funciona%20con%20el%20%C3%A1rbol%20de%20objetos.*

3. *Patrón Composite: definición, diagrama UML y ejemplos. (2020, November 9). IONOS Digital Guide. https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/patron-composite/*